

Malware Analysis Report

Reverse Engineering and Behavioral Analysis of an AgentTesla-Tagged Infostealer Sample

Disclaimer

This report was prepared for educational purposes, malware analysis, and defensive security research. The sample was analyzed inside an isolated virtual machine (VM) in a fully offline environment without an Internet connection. This report does not encourage the deployment, distribution, or unauthorized use of malware. The analysis focused on understanding the sample's structure, execution flow, and observable behavior. The information is intended to support malware research, threat detection, incident response, and defensive security practices.

1. Executive Summary

This case study presents the reverse engineering and behavioral analysis of an AgentTesla infostealer sample obtained from MalwareBazaar. The analysis focused on identifying the packed layer, reaching the Original Entry Point (OEP), recovering the payload, and comparing the behavior of the packed sample with the unpacked payload.

The analysis included PE inspection, static and dynamic analysis, manual unpacking, OEP identification, IAT reconstruction, and behavioral monitoring using Process Monitor.

During debugging, an address relocation / ASLR-related issue caused access violations. After adjusting the relevant PE header setting in the controlled analysis copy, the debugging environment became more stable, allowing the execution flow to be followed toward the suspected OEP.

The sample showed characteristics consistent with a packed executable, including a high entropy of 7.934, an obfuscated execution flow, and significant file-writing activity in AppData and Temp. Temporary artifacts such as tapestrylike.exe and aut8E90.tmp were also observed.

The suspected OEP was successfully reached and the memory image was dumped. Scylla recovered 527 valid APIs with 0 missed APIs, followed by successful import reconstruction. The entropy of the resulting dump decreased from 7.934 to 5.911, supporting the conclusion that a substantial portion of the packed layer had been removed.

Further static analysis in IDA Pro exposed clearer references to registry interaction, file management, and other Windows API functionality. Behavioral comparison also showed that the original sample generated approximately 1.62 MB of file-write activity, while the dumped payload did not reproduce the same large file-dropping behavior and instead mainly showed registry-reading activity.

No network activity was observed because the analysis environment was fully offline. The dumped payload also terminated with a non-zero exit status, suggesting that additional environment checks or runtime dependencies may still be present.

Overall, the analysis successfully exposed and validated the underlying payload, while also demonstrating that a successfully unpacked sample may still require further analysis before it can operate independently under different laboratory conditions.

2. Sample Information

The analyzed sample was obtained from MalwareBazaar and identified as an AgentTesla sample.

Attribute	Description
MD5	ac4ab3c4b9386b0355d8645f77f91e3e
SHA-256	363da150d891da7bb5da8056414882429067a0fcb27f58363567567bf18a323e
Sample Source	MalwareBazaar
Malware Type	Infostealer / Trojan
Architecture	32-bit

2.1 Sample Classification Note

The sample was identified as AgentTesla based on the available MalwareBazaar information. AgentTesla is commonly associated with information-stealing malware that can target sensitive data stored on or entered into an infected system.

The classification provided by MalwareBazaar should be treated as an initial identification reference rather than definitive proof of the sample's identity or capabilities. Repository labels and antivirus detection names can differ, and the actual behavior of a sample may vary depending on its configuration and execution environment.

For this reason, this report uses AgentTesla as the working malware classification based on the available sample information. Further confirmation should rely on additional evidence, such as verified malware signatures, static code analysis, observed behavior, and comparison with trusted AgentTesla samples.

3. Analysis Environment

The analysis was performed inside a controlled and isolated malware-analysis environment.

The main tools used were:

- DIE (Detect It Easy) — initial identification of executable characteristics and possible packer/runtime indicators.
- PEStudio — PE structure and suspicious characteristic inspection.
- IDA Pro — static analysis, function identification, cross-reference analysis, and code validation.
- x32dbg — dynamic debugging and execution tracing.
- Scylla — import discovery, dumping, and IAT reconstruction.
- Process Monitor (ProcMon) — monitoring file system, registry, process, and other observable system activity.

The test environment was fully offline. This was intentional to prevent the sample from communicating with external infrastructure.

As a result, network-related behavior observed during this case study should not be interpreted as proof that the malware has no network capability. The absence of network activity was expected because the environment had no Internet connectivity.

4. Initial PE Inspection

Before starting the dynamic analysis, the sample was inspected using DIE and PEStudio. The initial inspection showed characteristics consistent with a protected or packed executable. The original sample also had a relatively high entropy value of approximately 7.934

High entropy can be an indication that a file contains compressed, encrypted, or otherwise obfuscated data. In this case, the high entropy combined with the observed execution behavior suggested that the initial executable did not directly expose its complete payload.

During the early execution stage, the sample was also observed interacting with temporary files, including artifacts such as:

- `tapestrylike.exe`
- `aut8E90.tmp`

These observations became important later when comparing the behavior of the original packed sample with the unpacked payload.

5. Pre-Analysis Hardening and Address Relocation Issue

5.1 Initial Debugging Problem

During the first debugging attempts, the sample did not behave consistently under x32dbg. An access violation exception was observed during execution, with an invalid or unexpected pointer being involved in the failing instruction.

The issue appeared to be related to the way the executable was being mapped into memory and the presence of dynamic base / ASLR-related behavior. This created an unstable debugging environment and made it difficult to reliably follow the execution flow toward the original entry point.

5.2 PE Header Adjustment

To make the controlled laboratory analysis more stable, the PE header was inspected using CFF Explorer. The "DLL can move" option was disabled in the PE characteristics. The purpose of this change was not to modify the malware's functionality, but to reduce address variability during the controlled debugging process.

After the adjustment, the executable could be debugged more consistently and the previous access violation problem was no longer observed during the relevant execution path. This provided a more stable environment for continuing the unpacking analysis.

6. Manual Unpacking Methodology

The next stage was to identify where the packed layer transferred control to the actual payload. The general workflow was:

- a. Inspect the original executable.
- b. Identify suspicious execution flow.
- c. Trace the execution through the unpacking stage.
- d. Identify the suspected Original Entry Point.
- e. Dump the process memory at the appropriate stage.
- f. Reconstruct the imports.
- g. Validate the resulting executable using static and behavioral analysis.

7. Original Entry Point Discovery

During static analysis in IDA Pro, a jump associated with the transition toward the original entry point was identified. The relevant address calculation was documented as $0x517BBD - 0x400000 = 0x117BBD$.

The resulting RVA was then used together with the observed memory mapping in x32dbg. The corresponding runtime address was calculated as $0x400000 + 0x117BBD = 0x517BBD$. During

the debugging process, the execution flow was followed until the suspected original payload entry point was reached.

8. Reaching the OEP

Using x32dbg, execution was controlled with standard debugger stepping and execution controls. The analysis followed the execution flow until the suspected Original Entry Point (OEP) was reached.

The OEP used during the dumping stage was documented as 00784E87. Reaching this point was important because dumping the executable too early could result in capturing only the packed stub instead of the unpacked payload. At this stage, the memory image was considered suitable for the next phase of the analysis.

9. Dumping and IAT Reconstruction

After reaching the suspected OEP, Scylla was used to reconstruct the executable's imports. The following parameters were recorded during the process:

Parameter	Value
OEP	00434E87
VA	0048F000
Size	00000880

Scylla's import search successfully identified:

- 527 valid APIs
- 0 missed APIs

The process then produced a dumped executable followed by an import reconstruction step. The resulting file was saved as `_scy.exe`

This was an important validation point because a dumped PE file is not necessarily immediately executable or analyzable. Reconstructing the import information allowed the dumped image to be examined as a more complete PE file.

10. Entropy Comparison

One of the clearest indicators of successful unpacking was the difference in entropy between the original sample and the dumped payload.

File	Entropy
Original sample	7.934
_scy.exe	5.911
Difference	2.023

The entropy decreased by approximately 2.023 points. This large reduction is consistent with the removal of a high-entropy packed or obfuscated layer. However, entropy alone should not be treated as proof of successful unpacking. It becomes much more meaningful when combined with:

- reaching the suspected OEP,
- successful memory dumping,
- successful IAT reconstruction,
- static code validation,
- and behavioral comparison.

In this case, these indicators supported the conclusion that the packed layer had been substantially removed.

11. Static Analysis of the Unpacked Payload

The dumped executable was opened in IDA Pro for further validation. Compared with the original packed file, the dumped sample provided a much clearer view of the executable's internal code and imported functionality.

The analysis exposed references related to capabilities commonly associated with an infostealer, including:

- Registry interaction
- File management
- System information access
- Windows API usage
- Network-related functionality

The unpacked structure made it easier to identify and follow these areas without having to first pass through the original protection or unpacking layer. This provided additional evidence that `_scy.exe` represented the underlying payload rather than simply another copy of the original packing stub.

12. Behavioral Analysis

Behavioral analysis was performed using Process Monitor (ProcMon). The main purpose was to compare the behavior of:

- a. The original packed sample
- b. The dumped payload

This comparison was particularly useful because the two files did not produce the same observable activity.

12.1 Behavior of the Original Packed Sample

The original sample generated significant file-system activity during execution. One of the most noticeable observations was a large amount of data being written to locations under:

- AppData
- Temp

A total write activity of approximately 1,620,996 bytes, was observed in the relevant monitoring period. Artifacts such as:

- tapestrylike.exe
- aut8E90.tmp

were also observed.

This behavior is consistent with a packed execution stage in which the initial executable prepares, extracts, or reconstructs additional executable content before transferring execution to another stage.

Therefore, the large file-writing activity should not automatically be interpreted as the main infostealing functionality. Instead, it appears to be closely related to the sample's initial execution or unpacking stage.

13. Behavior of the Unpacked Payload

The dumped payload, _scy.exe, behaved differently. During the offline execution test, it did not reproduce the same large file-dropping behavior observed from the original packed sample. The monitored file-write activity relevant to the previous dropping behavior was effectively 0 bytes.

This difference was important because it supported the idea that the large file-writing activity from the original executable belonged primarily to the packed/staging layer rather than the core payload.

13.1 Registry Activity

The unpacked payload showed multiple registry-read operations. Observed locations included:

- HKCU\Control Panel\Desktop
- HKCU\Software\Classes
- HKCU\Software\Policies
- HKLM\OSDATA
- HKLM\SOFTWARE\Policies
- HKLM\SOFTWARE\WOW6432Node
- HKLM\System\CurrentControlSet

Registry access can be used by malware for several reasons, including:

- collecting system information,
- checking the Windows environment,
- identifying configuration settings,
- checking security-related policies,
- detecting execution conditions,

The observed registry activity therefore provides useful behavioral evidence about the payload's interaction with the host environment. However, registry reads alone are not sufficient to determine the exact purpose of every query. Each access should be correlated with the corresponding code path during deeper reverse engineering.

14. Network Behavior

No network traffic was observed during the test. This result was expected because the analysis environment was fully isolated and offline.

Therefore, no network activity does not mean that the AgentTesla sample has no network communication capability. It only means that network communication was not observed under the specific laboratory conditions used in this case study.

15. Anti-Analysis and Exit Status Observation

During validation of `_scy.exe`, the dumped payload terminated with a non-zero exit status. The process did not crash the debugger or cause the same access violation observed during the initial analysis.

This behavior suggests that the payload may still depend on runtime conditions that were not available in the isolated environment. Possible explanations include:

- environment checks,
- anti-debugging logic,
- anti-analysis checks,
- expected runtime dependencies,

- execution-state assumptions,
- missing components,
- or other validation routines inside the payload.

At this stage, this should be treated as an analysis hypothesis rather than a confirmed anti-debugging mechanism.

16. Packed vs. Unpacked Behavioral Comparison

The following table summarizes the main differences observed during the analysis.

Behavior	Original Packed Sample	Unpacked Payload
Large file writes	Observed	Not reproduced
AppData / Temp activity	Observed	Limited / not observed
Temporary executable artifacts	tapestrylike.exe, aut8E90.tmp	Not observed
Registry activity	Present	Clearly observed
Network activity	None	None
Environment checks	Suspected	Possible
Exit behavior	Normal execution stage	Non-zero exit status
Debugging stability	Required address adjustment	More stable
Entropy	7.934	5.911
Import visibility	Obfuscated/limited initially	527 APIs recovered

The behavioral difference provides useful evidence that the original executable contains more than just the core payload.

17. Indicators of Compromise

The following indicators were collected during this case study.

- a. MD5: ac4ab3c4b9386b0355d8645f77f91e3e
- b. SHA-256: 363da150d891da7bb5da8056414882429067a0fcb27f58363567567bfl8a323e
- c. Observed File Artifacts
 - tapestrylike.exe
 - aut8E90.tmp
 - _scy.exe

The first two artifacts were observed during the original sample's execution and should be treated as sample-specific artifacts, rather than universal AgentTesla filenames.

18. Detection and Mitigation Recommendations

Organizations should not rely on a single indicator to detect this type of threat. A layered approach is recommended.

18.1 Endpoint Monitoring

Monitor suspicious processes that:

- create executable files inside user-writable directories,
- execute newly created files from Temp or AppData,
- create unexpected temporary executables,
- perform unusual registry enumeration,
- or show abnormal parent-child process relationships.

18.2 File-System Monitoring

Security teams should pay particular attention to executable creation in locations such as:

- %TEMP%
- %APPDATA%

- %LOCALAPPDATA%

Especially suspicious behavior includes a process creating an executable in a temporary directory and immediately executing it.

18.3 Application Control

Where possible, restrict execution from user-writable directories. Application allowlisting can significantly reduce the impact of malware that attempts to drop and execute secondary payloads from locations such as Temp or AppData.

18.4 Network Controls

Even though no network traffic was observed in this isolated case, AgentTesla should be treated as potentially capable of external communication. Organizations should therefore maintain:

- outbound traffic monitoring,
- DNS monitoring,
- proxy logging,
- TLS inspection where appropriate,
- and endpoint-to-Internet behavioral detection.

Network isolation during incident response can also help prevent credential or information theft from continuing.

18.5 Credential Protection

Because AgentTesla is an infostealer, organizations should assume that credentials and other sensitive information may be at risk when an infection is confirmed. Recommended actions include:

- Isolate the affected endpoint.
- Identify potentially compromised accounts.

- Reset affected credentials from a known-clean device.
- Revoke active sessions where appropriate.
- Enable MFA.

19. Limitations

This case study has several important limitations.

First, the analysis was performed in a fully offline environment. Therefore, network communication and command-and-control behavior could not be observed.

Second, the dumped payload exited with a non-zero status. This means that some runtime functionality may not have been fully exercised.

Third, the behavioral observations represent only the execution path triggered during the specific laboratory tests. Malware may contain functionality that is activated only under certain conditions.

Finally, the observed file names and registry operations should not be considered universal indicators for every AgentTesla sample. Malware variants and builds can behave differently.

20. Conclusion

This case study demonstrated a practical workflow for analyzing a packed AgentTesla infostealer through static analysis, dynamic debugging, manual unpacking, import reconstruction, and behavioral comparison.

The initial sample showed characteristics consistent with a packed executable, including a high entropy value of 7.934 and significant file-writing activity involving temporary locations.

An address-relocation issue initially caused instability during debugging. After adjusting the relevant PE header characteristic in the controlled analysis copy, the debugging environment became stable enough to continue the execution analysis.

The suspected OEP was reached, allowing the process memory to be dumped and analyzed with Scylla. Import reconstruction was successful, with 527 valid APIs and 0 missed APIs.

The resulting dump also showed a significant entropy reduction $7.934 \rightarrow 5.911$.

Together with successful OEP identification, dumping, IAT reconstruction, and static validation, these results provide strong evidence that the packed layer was substantially removed.

The behavioral comparison was particularly useful. The original packed sample generated approximately 1.62 MB of file-write activity and created temporary executable artifacts, while the unpacked payload did not reproduce the same large dropping behavior. Instead, the payload showed multiple registry interactions and terminated with a non-zero exit status.

The final result demonstrates an important point in malware analysis, successful unpacking does not necessarily mean that every payload behavior will immediately work in an isolated environment. Runtime checks, environmental dependencies, and anti-analysis mechanisms may still affect execution.

Overall, the analysis successfully progressed from an opaque packed executable to a significantly more transparent payload that could be examined through IDA Pro and correlated with observable system behavior. The findings provide useful material for malware reverse engineering, behavioral analysis, detection engineering, and defensive security research.